CS & IT ENGINEERING



COMPUTER ORGANIZATION AND ARCHITECTURE

Pipelinine Processing

Lecture No.- 5



By- Vishvadeep Gothi sir

Recap of Previous Lecture



- **Topic** Instruction Pipeline
- **Topic** Data Hazard Classification
- **Topic** RAW, WAW, WAR

Topics to be Covered



Topic CPI in Instruction Pipeline

Topic Classical RISC Pipeline

[NAT]

#Q. Consider the following sequence of instructions, which is to be executed on a 5 stage pipeline: IF, ID, OF, EX and WB. Each stage takes one cycle for each instruction. Total Cycles required to execute this sequence

1. When pipeline uses operand forwarding $(5+6-1)+(5-3) = 12$
2. When pipeline does not use operand forwarding $(5+6-1)+2(5-3) = 14$

ADD R2, R1, R0	R2←R1+R0
MUL R4, R3, R2	R4←R3*R2
LOAD R7, (1000)	R7←M[1000]
ADD R6, R7, R8	R6←R7+R8
MUL R7, R1, R2	R7←R1*R2
SUB R1, R3, R4	R1←R3-R4

stalls

[NAT]

GATE- PYQ

#Q. Consider a non-pipelined processor operating at 2.5 GHz. It takes 5 clock cycles to complete an instruction. You are going to make a 5- stage pipeline out of this processor. Overheads associated with pipelining force you to operate the pipelined processor at 2 GHz. In a given program, assume that 30% are memory instructions, 60% are ALU instructions and the rest are branch instructions. 5% of the memory instructions cause stalls of 50 clock cycles each due to cache misses and 50% of the branch instructions cause stalls of 2 cycles each. Assume that there are no stalls associated with the execution of ALU instructions. For this program, the speedup achieved by the pipelined processor over the non-pipelined processor (round off to 2 decimal places) is 2.16 .

non-pipeline:-

cycle time $= \frac{1}{2.5\,GHz} = 0.4\,ns$

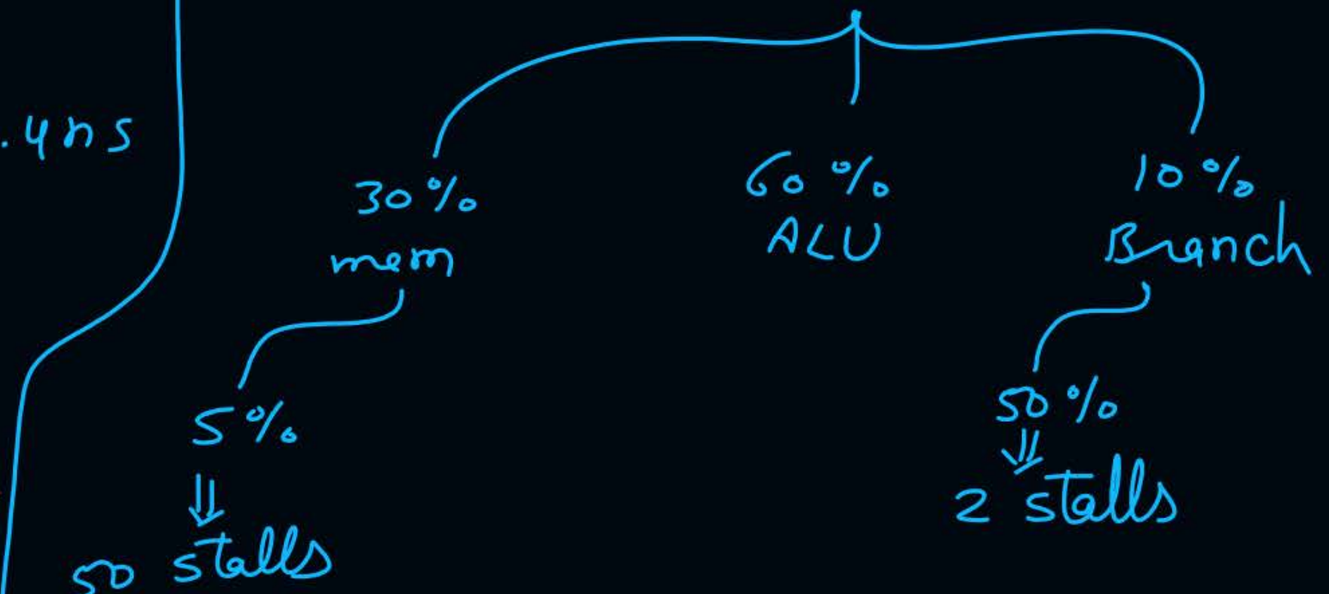
$t_n = 5 * 0.4\,ns$

$= 2\,ns$

pipeline:-

$k = 5$

$t_p = \frac{1}{2\,GHz} = 0.5\,ns$



$CPI = 1 + (0.3 * 0.05 * 50) + (0.1 * 0.5 * 2)$

$= 1.85$

$\text{speed up} = \frac{t_n}{CPI * t_p} = \frac{2\,ns}{1.85 * 0.5\,ns} = 2.162$

[NAT] GATE- PYQ

#Q. An instruction pipeline has five stages where each stage takes 2 nanoseconds, and all instructions use all five stages. Branch instructions are not overlapped, i.e., the instruction after the branch is not fetched till the branch instruction is completed. Under ideal conditions.

1. Calculate the average instruction execution time assuming that 20% of all instruction executed are branch instructions. Ignore the fact that some branch instructions may be conditional.

2. If a branch instruction is a conditional branch instruction, the branch need not be taken. If the branch is not taken, the following instructions can be overlapped. When 80% of all branch instructions are conditional branch instructions, and 50% of the conditional branch instructions are such that the branch is taken, calculate the average instruction execution time?

$t_p = 2ns$

stalls due to each branch inst^n = 5 − 1 = 4

1. CPI = 1 + 0.2 * 4 = 1.8

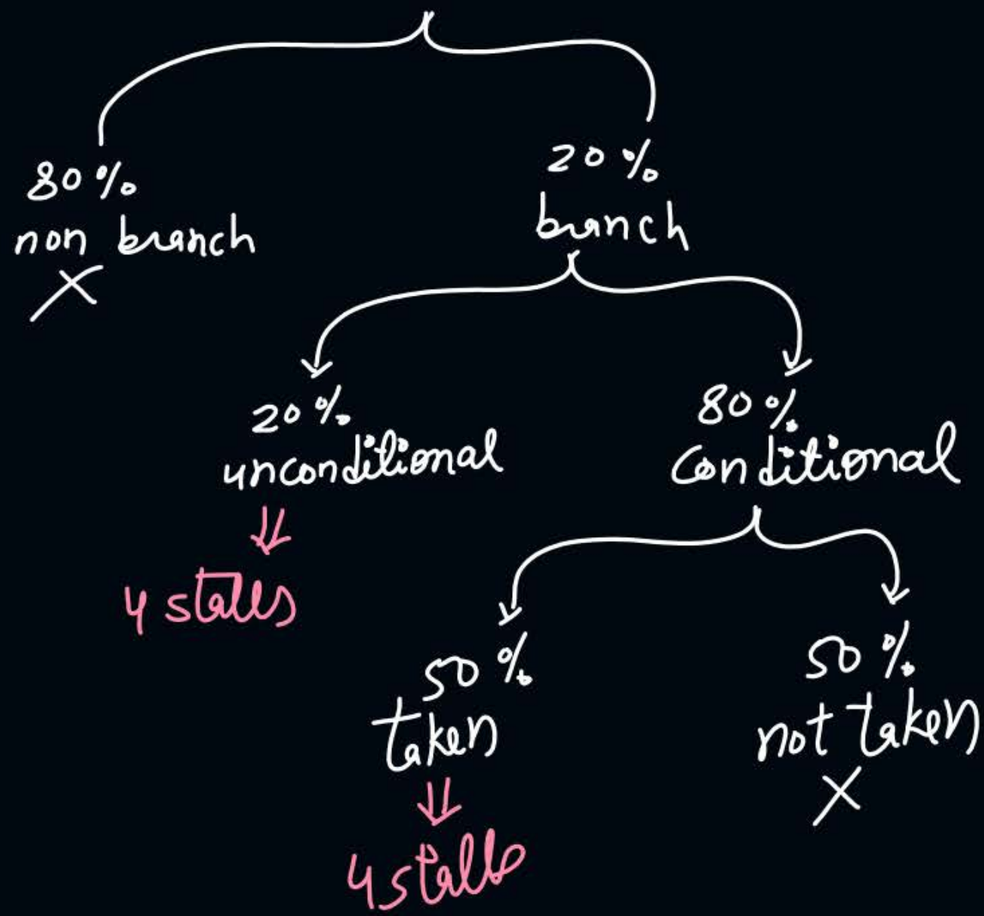
A.I.E.T. = 1.8 * 2 = 3.6 ns Ans

2. CPI = 1 + (0.2 * 0.2 * 4) + (0.2 * 0.8 * 0.5 * 4)

= 1.48

A.I.E.T. = 1.48 * 2ns

= 2.96 ns Ans.



[NAT]

Ans = 1.43

#Q. A processor X_1 operating at 2 GHz has a standard 5-stage RISC instruction pipeline having a base CPI (cycles per instruction) of one without any pipeline hazards. For a given program P that has 30% branch instructions, control hazards incur 2 cycles stall for every branch. A new version of the processor X_2 operating at same clock frequency has an additional branch predictor unit (BPU) that completely eliminates stalls for correctly predicted branches. There is neither any savings nor any additional stalls for wrong predictions. There are no structural hazards and data hazards for X_1 and X_2. If the BPU has a prediction accuracy of 80%, the speed up (rounded off to two decimal places) obtained by X_2 over X_1 in executing P is

X1 :-

$t_p = \frac{1}{2GHz} = 0.5\,ns$

$k = 5$

30% branch inst^n $\Rightarrow$ 2 stalls

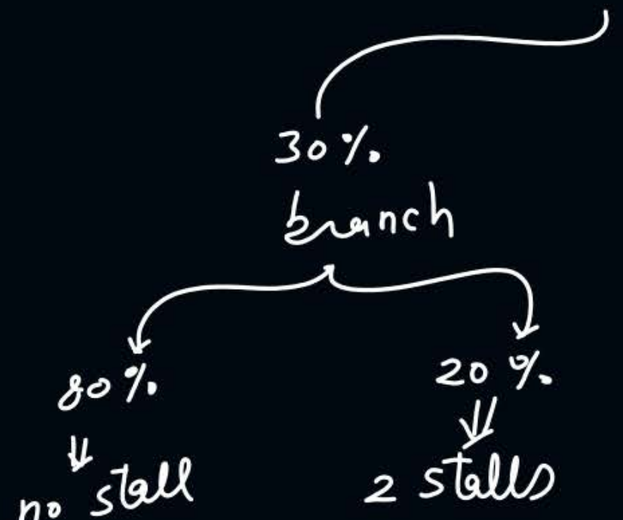
$CPI = 1 + 0.3 * 2$

$= 1.6$

$A.I.E.T. = 1.6 * 0.5\,ns$

X2 :-

$t_p = 0.5\,ns$



$CPI = 1 + 0.3 * 0.2 * 2$

$= 1 + 0.12$

$= 1.12$

$A.I.E.T. = 1.12 * 0.5\,ns$

$speed = \frac{1.6 * 0.5}{1.12 * 0.5} = 1.4285$

$= 1.43$

[NAT]

GATE - PYQ

Ans = 1.54

#Q. An instruction pipeline has five stages, namely, instruction fetch (IF), instruction decode and register fetch (ID/RF), instruction execution (EX), memory access (MEM), and register writeback (WB) with stage latencies 1 ns, 2.2 ns, 2 ns, 1 ns, and 0.75 ns, respectively (ns stands for nanoseconds). To gain in terms of frequency, the designers have decided to split the ID/RF stage into three stages (ID, RF1, RF2) each of latency 2.2/3 ns. Also, the EX stage is split into two stages (EX1, EX2) each of latency 1 ns. The new design has a total of eight pipeline stages. A program has 20% branch instructions which execute in the EX stage and produce the next instruction pointer at the end of the EX stage in the old design and at the end of the EX2 stage in the new design. The IF stage stalls after fetching a branch instruction until the next instruction pointer is computed. All instructions other than the branch instruction have an average CPI of one in both the designs. The execution times of this program on the old and the new design are P and Q nanoseconds, respectively. The value of P/Q is?

IF — 1 ns
ID/RF — 2.2 ns
EX — 2 ns
MEM — 1 ns
WB — 0.75 ns

k = 5

20% branch

stall = 3 - 1 = 2

CPI = 1 + 0.2 * 2
= 1.4

t_p = 2.2 ns

A.I.E.T. = 1.4 * 2.2 ns

IF — 1 ns
ID/RF1 — 0.73 ns
ID/RF2 — 0.73 ns
ID/RF3 — 0.73 ns
EX1 — 1 ns
EX2 — 1 ns
MEM — 1 ns
WB — 0.75 ns

k = 8

stall = 6 - 1 = 5

CPI = 1 + 0.2 * 5
= 2

t_p = 1 ns

A.I.E.T. = 2 * 1 = 2 ns

$$= \frac{1.4 * 2.2 \text{ ns}}{2 \text{ ns}}$$

= 1.54



Topic : Instruction Pipeline

IF: Instruction Fetch
ID: Instruction Decode & Address Calculation
OF: Operand Fetch
EX: Execution
WB: Write Back



Topic : Classic RISC Pipeline

1. IF — Instⁿ fetch
2. ID — Instⁿ decode & operand fetch (Reg. Read)
3. EX — Execution (ALU operation)
4. MEM — Mem. access
5. WB — write back to register



Topic : Classic RISC Pipeline for Computation

1. IF — Instⁿ fetch & PC increment
2. ID — Instⁿ decode and operand fetch from register
3. EX — Perform ALU operation
4. MEM — ×
5. WB — Write back result to destination Register

By this standard,
stalls due to data hazard
= 5 − 2 = 3



Topic : Classic RISC Pipeline for Load

Reg ← Mem.

1. IF — Instⁿ fetch & PC inc.
2. ID — Instⁿ decode
3. EX — ×
4. MEM — mem. Read
5. WB — Reg. write



Topic : Classic RISC Pipeline for Store

Mem. ← Reg.

1. IF —— Instⁿ fetch & PC inc.
2. ID —— Instⁿ decode and reg. read
3. EX —— X
4. MEM —— mem. write
5. WB —— X



Topic : Classic RISC Pipeline for Branch

1. IF — Instⁿ fetch & PC increment
2. ID — Instⁿ decode, Target add. calculation, Condition check, PC updation appropriately.
3. EX — ×
4. MEM — ×
5. WB — ×

Stalls due to each branch instⁿ = 2 − 1

= 1

[NAT]

#Q. Consider a program which contains 50 instructions I1, I2 , I3 ..... I50. Further consider a 5-stage classical RISC pipeline with stages as: IF, ID, EX, MEM and WB. The program contains only 1 branch instruction which is instruction I5, and its target is instruction I48. If during the execution of the program the branch is taken, then number of cycles required to execute this program in the given pipeline is 13 ?

stall due to branch = 2 − 1
= 1

no. of instns executed (n) = 5 + 3 = 8

w/o hazard = 5 + 8 − 1 = 12
stall = 1
Total = 13

[NAT] GATE- 22

#Q. Consider a pipelined processor with 5 stages, Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write Back (WB). Each stage of the pipeline, except the EX-stage, takes one cycle. Assume that the ID stage merely decodes the instruction and the register read is performed in the EX-stage. The EX-stage takes one cycle for ADD instruction and two cycles for MUL instruction. Ignore pipeline register latencies.

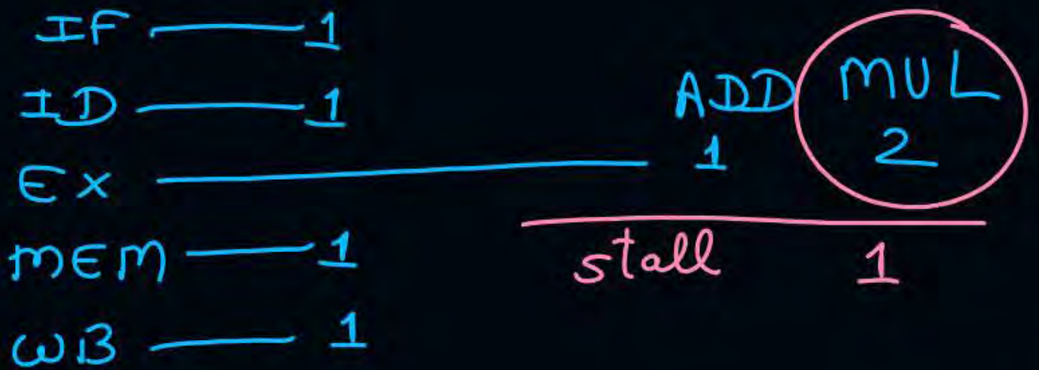
Consider the following sequence of 8 instructions:

ADD, MUL, ADD, MUL, ADD, MUL, ADD, MUL

Assume that every MUL instruction is data-dependent on the ADD instruction just before it and every ADD instruction (except the first ADD) is data-dependent on the MUL instruction just before it. The Speedup is defined as follows:

$$Speed\ up = \frac{Execution\ time\ without\ operand\ forwarding}{Execution\ time\ with\ operand\ forwarding} = \frac{30}{16} = 1.875 = 1.88$$

The Speedup achieved in executing the given instruction sequence on the pipelined processor (rounded to 2 decimal places) is 1.88.



Ex ⇒ operand fetch

no. of stalls due to each data hazard = 5 − 3 = 2

with operand forw.

n = 8
k = 5

w/o stall = 5 + 8 − 1 = 12

stalls for structural hazard = 4 * 1 = 4

Total = 16

---

w/o operand forwarding :-

w/o stall = 5 + 8 − 1 = 12

structural hazard stalls = 4 * 1 = 4

data hazard stalls = 7 * 2 = 14

Total = 30



2 mins Summary

Topic CPI in Instruction Pipeline

Topic Classical RISC Pipeline



Happy Learning

THANK - YOU